

École Nationale Supérieure d'Informatique et
d'Analyse des Systèmes
ENSIAS

Rapport d'Avancement

DevPilot AI

Plateforme Agentic RAG pour l'Exploitation Intelligente des
Documents Privés

Réalisé par : Oussama Mahi Bilal MALIH
Projet : DevPilot AI
Sujet : Embeddings, recherche sémantique et chat RAG
Technologies : Ollama, Qdrant, FastAPI, PostgreSQL, Celery, Redis

Année universitaire 2025–2026

Table des matières

Résumé	3
1 Contexte général	4
1.1 Rappel de l'objectif du projet	4
1.2 État avant la Phase 9	4
1.3 Objectif des phases 9 à 11	5
2 Architecture RAG actuelle	6
2.1 Vue globale	6
2.2 Rôle des composants	6
2.3 Approche Ollama-first	7
3 Phase 9 : Embeddings avec Ollama et indexation Qdrant	8
3.1 Objectif de la Phase 9	8
3.2 Principe des embeddings	8
3.3 Modèle utilisé	8
3.4 Stockage dans Qdrant	8
3.5 Workflow de la Phase 9	9
3.6 Résultat obtenu	9
3.7 Difficultés rencontrées	10
3.8 Validation de la Phase 9	10
4 Phase 10 : Recherche sémantique	11
4.1 Objectif de la Phase 10	11
4.2 Principe de la recherche sémantique	11
4.3 Endpoint ajouté	11
4.4 Résultat attendu	11
4.5 Sécurité et isolation des workspaces	12
4.6 Test d'isolation	12
4.7 Validation de la Phase 10	12
5 Phase 11 : Chat RAG avec Ollama et citations	13
5.1 Objectif de la Phase 11	13
5.2 Workflow de la Phase 11	13
5.3 Endpoint ajouté	14
5.4 Prompt RAG utilisé	14
5.5 Résultat obtenu	14
5.6 Analyse du résultat	15
5.7 Gestion des questions hors contexte	15

5.8	Validation de la Phase 11	15
6	Configuration Ollama et modèles	16
6.1	Problème rencontré	16
6.2	Analyse	16
6.3	Correction réalisée	16
6.4	Choix des modèles	17
6.5	Configuration recommandée	17
7	Tests réalisés	18
7.1	Test de disponibilité d'Ollama	18
7.2	Test d'accès à Ollama depuis Docker	18
7.3	Test de génération directe	18
7.4	Test de recherche sémantique	19
7.5	Test du chat RAG	19
7.6	Test de sauvegarde des messages	19
7.7	Test de sauvegarde des chunks récupérés	20
8	État actuel après la Phase 11	21
8.1	Avancement estimé	21
9	Difficultés rencontrées et solutions	22
9.1	Connexion entre Docker et Ollama	22
9.2	Conflit de processus Ollama	22
9.3	Lenteur du modèle de génération	22
9.4	Tokens JWT expirés	22
10	Prochaine étape : Phase 12	24
10.1	Objectif de la Phase 12	24
10.2	Pipeline prévu	24
10.3	Importance de cette phase	24
	Conclusion	25

Résumé

Ce rapport présente l'avancement du projet **DevPilot AI** durant les phases 9 à 11. Ces phases marquent une évolution importante du projet, car elles transforment le pipeline documentaire déjà construit en une véritable plateforme RAG fonctionnelle.

Les phases précédentes ont permis de construire la base backend : authentification, workspaces, upload de documents, traitement asynchrone, extraction de texte, nettoyage et découpage en chunks. Les phases 9 à 11 ajoutent maintenant l'intelligence documentaire : génération d'embeddings avec Ollama, stockage vectoriel dans Qdrant, recherche sémantique, puis génération de réponses avec citations.

À la fin de la phase 11, le système est capable de recevoir une question utilisateur, de retrouver les chunks les plus pertinents dans Qdrant, de construire un contexte, d'interroger un modèle local via Ollama, puis de retourner une réponse accompagnée de citations.

1 Contexte général

1.1 Rappel de l'objectif du projet

DevPilot AI est une plateforme de type *Retrieval-Augmented Generation*, ou RAG, destinée à l'exploitation intelligente de documents privés. L'objectif est de permettre à un utilisateur de téléverser ses documents, puis de poser des questions en langage naturel afin d'obtenir des réponses basées uniquement sur ces documents.

Le projet vise à combiner plusieurs dimensions importantes :

- ingénierie backend ;
- sécurité et isolation des données ;
- traitement asynchrone ;
- indexation vectorielle ;
- recherche sémantique ;
- génération de réponses avec citations ;
- utilisation de modèles locaux avec Ollama.

1.2 État avant la Phase 9

Avant la phase 9, le système permettait déjà :

- l'inscription et la connexion des utilisateurs ;
- la gestion des workspaces ;
- l'upload de documents PDF, TXT et Markdown ;
- le traitement asynchrone avec Celery et Redis ;
- l'extraction du texte ;
- le nettoyage du texte ;
- le découpage en chunks ;
- le stockage des chunks dans PostgreSQL.

Cependant, les chunks étaient encore uniquement stockés sous forme textuelle. Ils n'étaient pas encore transformés en vecteurs. La recherche sémantique et le chat RAG n'étaient donc pas encore opérationnels.

1.3 Objectif des phases 9 à 11

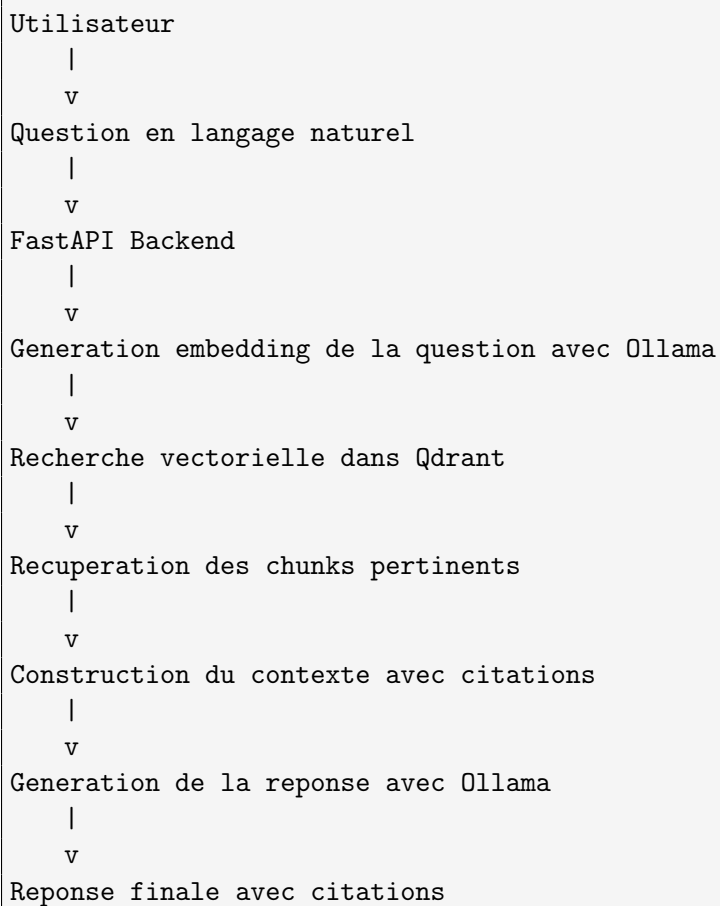
Les phases 9 à 11 ont pour objectif de construire le coeur du système RAG :

- Phase 9 : transformer les chunks en embeddings et les stocker dans Qdrant ;
- Phase 10 : permettre la recherche sémantique dans les documents ;
- Phase 11 : générer des réponses à partir des chunks récupérés, avec citations.

2 Architecture RAG actuelle

2.1 Vue globale

L'architecture RAG actuelle peut être résumée ainsi :



2.2 Rôle des composants

Composant	Rôle dans les phases 9 à 11
FastAPI	Expose les endpoints de recherche et de chat RAG
PostgreSQL	Stocke les documents, chunks, conversations, messages et citations
Ollama	Génère les embeddings et les réponses localement

Qdrant	Stocke les vecteurs et effectue la recherche sémantique
Celery	Traite les documents en arrière-plan et indexe les chunks
Redis	Sert de broker de messages pour Celery
JWT	Protège les endpoints et identifie l'utilisateur courant

2.3 Approche Ollama-first

Le projet adopte une approche **Ollama-first**. Cela signifie que les modèles d'intelligence artificielle sont exécutés localement, sans dépendre obligatoirement d'une API payante externe.

Cette approche présente plusieurs avantages :

- réduction des coûts ;
- meilleure confidentialité des données ;
- possibilité de travailler hors ligne ou en environnement local ;
- contrôle direct sur les modèles utilisés ;
- adaptation aux contraintes matérielles de la machine.

Dans l'état actuel, le modèle d'embedding utilisé est :

```
nommic-embed-text
```

Le modèle de génération peut être configuré selon les ressources disponibles :

```
qwen3:8b  
qwen2.5:3b  
qwen2.5:3b-instruct-q3_K_S
```


3 Phase 9 : Embeddings avec Ollama et indexation Qdrant

3.1 Objectif de la Phase 9

La Phase 9 consiste à transformer les chunks textuels créés durant la Phase 8 en vecteurs numériques appelés **embeddings**. Ces vecteurs sont ensuite stockés dans la base vectorielle Qdrant.

L'objectif est de permettre une recherche basée sur le sens du texte, et non uniquement sur des mots-clés exacts.

3.2 Principe des embeddings

Un embedding est une représentation numérique d'un texte. Deux textes ayant un sens proche auront des vecteurs proches dans l'espace vectoriel.

Par exemple, les phrases suivantes peuvent être proches sémantiquement :

```
How does DevPilot AI process documents?  
How are documents handled by the platform?
```

Même si les mots ne sont pas exactement les mêmes, la recherche vectorielle peut comprendre que les deux phrases ont un sens similaire.

3.3 Modèle utilisé

Le modèle utilisé pour générer les embeddings est :

```
omic-embed-text
```

Ce modèle est adapté à la génération d'embeddings textuels. Il est utilisé uniquement pour transformer les documents et les questions en vecteurs. Il ne doit pas être utilisé pour générer des réponses conversationnelles.

3.4 Stockage dans Qdrant

Qdrant est utilisé pour stocker les vecteurs générés à partir des chunks. Chaque point stocké dans Qdrant contient :

- le vecteur numérique ;
- l'identifiant du chunk ;

- l'identifiant du document ;
- l'identifiant du workspace ;
- l'index du chunk ;
- le nom du fichier.

Le nom de la collection utilisée est :

```
devpilot_chunks
```

3.5 Workflow de la Phase 9

Le workflow de la Phase 9 est le suivant :

```
Document upload
  |
  v
Celery task
  |
  v
Extraction + cleaning + chunking
  |
  v
Generation embeddings avec Ollama
  |
  v
Creation collection Qdrant si necessaire
  |
  v
Insertion des vecteurs dans Qdrant
  |
  v
Sauvegarde du vector_id dans PostgreSQL
  |
  v
Document status = indexed
```

3.6 Résultat obtenu

Après correction de la connectivité entre Docker et Ollama, les chunks sont bien transformés en vecteurs et indexés dans Qdrant.

La preuve principale est que la colonne `vector_id` dans la table `chunks` n'est plus vide.

Commande de vérification :

```
docker compose exec postgres psql -U devpilot -d devpilot -c \
"select chunk_index, vector_id from chunks where document_id = '$DOCUMENT_ID'
order by chunk_index;"
```

Résultat attendu :

```
chunk_index | vector_id
-----+-----
0 | non-null vector identifier
```

3.7 Difficultés rencontrées

La difficulté principale de cette phase concernait la connexion entre les conteneurs Docker et Ollama installé sur la machine hôte.

Le backend et le worker Celery devaient appeler :

```
http://host.docker.internal:11434
```

Pour que cela fonctionne, Ollama doit écouter sur :

```
0.0.0.0:11434
```

et non uniquement sur :

```
127.0.0.1:11434
```

La configuration correcte utilisée est :

```
OLLAMA_HOST=0.0.0.0:11434
OLLAMA_MODELS=/home/oussama/.ollama/models
```

3.8 Validation de la Phase 9

La Phase 9 est considérée comme validée si :

- Ollama est accessible depuis le backend ;
- Ollama est accessible depuis le worker Celery ;
- Qdrant est accessible ;
- les embeddings sont générés avec `nomic-embed-text` ;
- les vecteurs sont stockés dans Qdrant ;
- `vector_id` est sauvegardé dans PostgreSQL ;
- le document passe au statut `indexed`.

4 Phase 10 : Recherche sémantique

4.1 Objectif de la Phase 10

La Phase 10 ajoute un endpoint de recherche sémantique. L'utilisateur peut envoyer une requête en langage naturel, et le système retourne les chunks les plus pertinents issus des documents de son workspace.

4.2 Principe de la recherche sémantique

La recherche sémantique fonctionne en trois étapes :

1. générer un embedding de la question utilisateur ;
2. rechercher dans Qdrant les vecteurs les plus proches ;
3. récupérer les chunks correspondants depuis PostgreSQL.

4.3 Endpoint ajouté

L'endpoint principal de cette phase est :

```
POST /api/v1/workspaces/{workspace_id}/retrieval/search
```

Exemple de requête :

```
curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/retrieval/
search \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "query": "What technologies does DevPilot AI use?",
  "top_k": 5
}'
```

4.4 Résultat attendu

Le système doit retourner une liste de chunks pertinents :

```
{
  "results": [
    {
      "chunk_id": "...",
```

```
    "document_id": "...",
    "filename": "...",
    "content": "...",
    "chunk_index": 0,
    "score": 0.70,
    "metadata": {}
  }
]
```

4.5 Sécurité et isolation des workspaces

La recherche sémantique respecte l'isolation des workspaces. Un utilisateur ne peut rechercher que dans les documents appartenant à ses propres workspaces.

Cela est assuré par :

- la vérification du JWT ;
- la vérification de l'appartenance au workspace ;
- le filtrage par `workspace_id` dans Qdrant ;
- la récupération des chunks uniquement depuis le workspace autorisé.

4.6 Test d'isolation

Un deuxième utilisateur est créé pour tester l'isolation. Lorsqu'il tente d'appeler l'endpoint de recherche sur un workspace qui ne lui appartient pas, le système doit retourner :

```
403 Forbidden
```

ou :

```
404 Not Found
```

4.7 Validation de la Phase 10

La Phase 10 est considérée comme validée si :

- la recherche retourne des chunks pertinents ;
- les scores de similarité sont présents ;
- le paramètre `top_k` est respecté ;
- les résultats appartiennent au bon workspace ;
- un autre utilisateur ne peut pas accéder aux résultats ;
- la recherche utilise les vecteurs stockés dans Qdrant.

5 Phase 11 : Chat RAG avec Ollama et citations

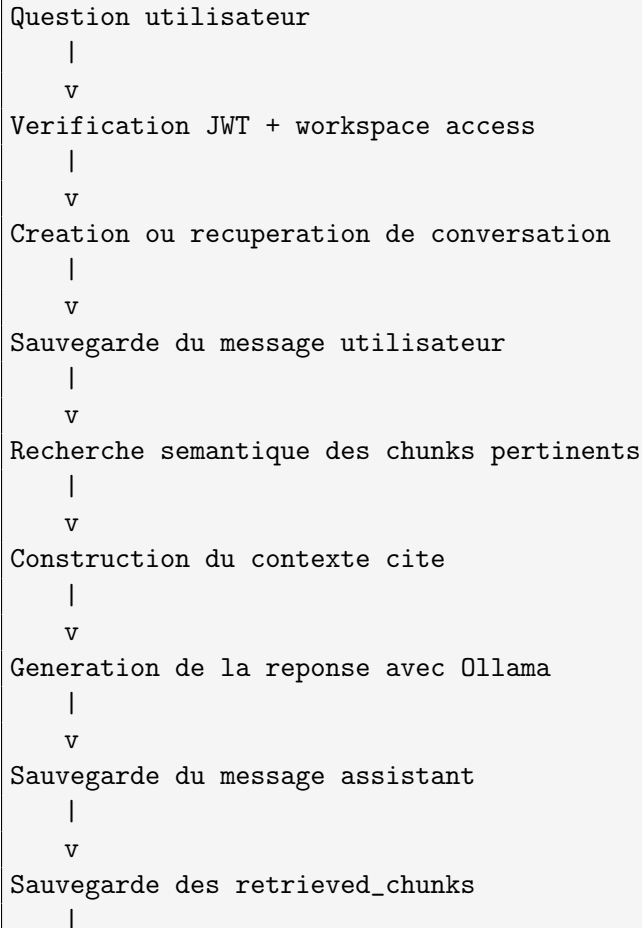
5.1 Objectif de la Phase 11

La Phase 11 transforme la recherche sémantique en une véritable expérience de chat RAG.

L'utilisateur pose une question, le système récupère les chunks pertinents, construit un contexte, appelle le modèle de génération via Ollama, puis retourne une réponse accompagnée de citations.

5.2 Workflow de la Phase 11

Le workflow est le suivant :



v
Retour de la reponse avec citations

5.3 Endpoint ajouté

L'endpoint principal est :

```
POST /api/v1/workspaces/{workspace_id}/chat/query
```

Exemple :

```
curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/chat/query \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "question": "What is DevPilot AI?"
}'
```

5.4 Prompt RAG utilisé

Le prompt de génération est conçu pour réduire les hallucinations. Le modèle doit répondre uniquement à partir du contexte fourni.

Les règles principales sont :

- répondre uniquement avec le contexte ;
- citer les sources avec [1], [2], [3] ;
- ne pas inventer d'informations ;
- dire que l'information n'existe pas si le contexte est insuffisant ;
- répondre dans la même langue que la question ;
- rester concis.

5.5 Résultat obtenu

Une question a été posée :

```
What is DevPilot AI?
```

Le système a retourné une réponse correcte :

```
DevPilot AI is a secure multi-workspace Agentic RAG platform for private
document intelligence [3].
It allows users to upload private PDF, TXT, and Markdown documents, process them
asynchronously with Celery and Redis, extract and clean text, split content
into chunks, generate embeddings with Ollama, store vectors in Qdrant,
retrieve relevant chunks, and generate answers with citations.
The backend uses FastAPI for APIs, PostgreSQL for metadata, Redis for queueing,
Celery for background jobs, Qdrant for vector search, and Ollama for local
AI models.
```

The platform protects private data using JWT authentication and workspace-level isolation [3].

Le résultat contient également :

- une réponse générée ;
- une liste de citations ;
- un `conversation_id` ;
- un `message_id`.

5.6 Analyse du résultat

Ce résultat est positif pour plusieurs raisons :

- la réponse est correcte ;
- la réponse est basée sur le document uploadé ;
- les citations sont retournées ;
- la conversation est sauvegardée ;
- le message assistant est sauvegardé ;
- les chunks récupérés sont traçables ;
- le système respecte l'objectif du RAG.

5.7 Gestion des questions hors contexte

Un bon système RAG ne doit pas répondre à des questions dont la réponse n'existe pas dans les documents fournis.

Par exemple, pour la question :

Who won the World Cup in 1998?

Le système doit répondre que l'information n'est pas disponible dans les documents uploadés, au lieu d'inventer une réponse.

5.8 Validation de la Phase 11

La Phase 11 est considérée comme validée si :

- l'utilisateur peut poser une question ;
- le système récupère les chunks pertinents ;
- le modèle Ollama génère une réponse ;
- la réponse contient des citations ;
- un `conversation_id` est retourné ;
- un `message_id` est retourné ;
- les messages sont sauvegardés dans PostgreSQL ;
- les chunks récupérés sont sauvegardés dans `retrieved_chunks` ;
- les questions hors contexte sont refusées proprement.

6 Configuration Ollama et modèles

6.1 Problème rencontré

Durant la Phase 11, le modèle `qwen3:8b` a été utilisé comme modèle de génération. Ce modèle donne de meilleurs résultats que des modèles plus petits, mais il peut être plus lent sur une machine locale.

Une erreur rencontrée était liée à l'indisponibilité ou au timeout du modèle :

```
Ollama generation is unavailable or timed out.  
Please check that Ollama is running and the selected model is installed: qwen3:8b.
```

6.2 Analyse

Ce problème peut être causé par :

- Ollama non démarré ;
- modèle `qwen3:8b` non installé ;
- timeout trop court ;
- conflit entre plusieurs processus Ollama ;
- backend incapable de joindre Ollama depuis Docker.

6.3 Correction réalisée

La configuration correcte d'Ollama pour l'environnement Docker est :

```
OLLAMA_HOST=0.0.0.0:11434  
OLLAMA_BASE_URL=http://host.docker.internal:11434
```

Le service Ollama doit être accessible depuis les conteneurs backend et Celery.

Commande de test :

```
docker compose exec celery_worker python -c \  
"import httpx; print(httpx.get('http://host.docker.internal:11434/api/tags',  
    timeout=5).text)"
```

6.4 Choix des modèles

Le modèle d'embedding reste :

```
nomic-embed-text
```

Le modèle de génération peut être :

```
qwen3:8b
```

pour une meilleure qualité, ou :

```
qwen2.5:3b
```

pour plus de rapidité.

6.5 Configuration recommandée

Pour une démonstration de qualité :

```
OLLAMA_GENERATION_MODEL=qwen3:8b
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
GENERATION_MAX_TOKENS=512
GENERATION_TEMPERATURE=0.2
EMBEDDING_DIMENSION=768
```

Pour un développement rapide :

```
OLLAMA_GENERATION_MODEL=qwen2.5:3b
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
GENERATION_MAX_TOKENS=512
GENERATION_TEMPERATURE=0.2
EMBEDDING_DIMENSION=768
```

7 Tests réalisés

7.1 Test de disponibilité d'Ollama

```
curl http://localhost:11434/api/tags
```

Résultat attendu :

Liste des modeles disponibles dans Ollama

7.2 Test d'accès à Ollama depuis Docker

```
docker compose exec backend python -c \  
"import httpx; print(httpx.get('http://host.docker.internal:11434/api/tags',  
    timeout=5).text)"
```

```
docker compose exec celery_worker python -c \  
"import httpx; print(httpx.get('http://host.docker.internal:11434/api/tags',  
    timeout=5).text)"
```

Résultat attendu :

JSON contenant les modeles Ollama disponibles

7.3 Test de génération directe

```
curl http://localhost:11434/api/chat \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "qwen3:8b",  
  "messages": [  
    {  
      "role": "user",  
      "content": "Answer in one sentence: what is RAG?"  
    }  
  ],  
  "stream": false,  
  "options": {  
    "temperature": 0.2,  
    "num_predict": 128
```

```
}  
}'
```

Résultat attendu :

Une reponse courte expliquant le concept RAG

7.4 Test de recherche sémantique

```
curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/retrieval/  
search \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "query": "What technologies does DevPilot AI use?",  
  "top_k": 5  
'
```

Résultat attendu :

Liste de chunks pertinents avec score de similarite

7.5 Test du chat RAG

```
curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/chat/query \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "question": "What is DevPilot AI?"  
'
```

Résultat attendu :

Reponse generee avec citations, conversation_id et message_id

7.6 Test de sauvegarde des messages

```
docker compose exec postgres psql -U devpilot -d devpilot -c \  
"select role, left(content,120) as preview from messages order by created_at  
desc limit 10;"
```

Résultat attendu :

Messages utilisateur et assistant sauvegardes

7.7 Test de sauvegarde des chunks récupérés

```
docker compose exec postgres psql -U devpilot -d devpilot -c \  
"select message_id, chunk_id, score, rank from retrieved_chunks order by  
created_at desc limit 10;"
```

Résultat attendu :

Chunks utilisés pour générer les réponses sauvegardées avec score et rang

8 État actuel après la Phase 11

Module	État
Embeddings avec Ollama	Fonctionnel
Modèle d'embedding <code>omic-embed-text</code>	Fonctionnel
Qdrant vector store	Fonctionnel
Indexation des chunks dans Qdrant	Fonctionnelle
Sauvegarde du <code>vector_id</code>	Fonctionnelle
Recherche sémantique	Fonctionnelle
Filtrage par workspace	Fonctionnel
Endpoint retrieval/search	Fonctionnel
Chat RAG	Fonctionnel
Génération avec Ollama	Fonctionnelle
Citations	Fonctionnelles
Sauvegarde des conversations	Fonctionnelle
Sauvegarde des messages	Fonctionnelle
Sauvegarde des retrieved chunks	Fonctionnelle
Gestion des questions hors contexte	À renforcer par tests supplémentaires
Optimisation du modèle local	En cours

8.1 Avancement estimé

Après validation de la Phase 11, le projet peut être considéré comme ayant dépassé le simple stade d'un backend documentaire. Il possède maintenant un vrai pipeline RAG fonctionnel.

L'avancement estimé du MVP est :

```
Backend foundation : termine
Document ingestion : termine
Vector indexing : termine
Semantic retrieval : termine
RAG chat with citations : fonctionnel
Evaluation automatique : prochaine phase
Agentic workflow : prochaine phase
```

9 Difficultés rencontrées et solutions

9.1 Connexion entre Docker et Ollama

La difficulté la plus importante a été la connexion entre les conteneurs Docker et Ollama exécuté sur la machine hôte.

La solution consiste à utiliser :

```
host.docker.internal
```

et à configurer Ollama avec :

```
OLLAMA_HOST=0.0.0.0:11434
```

9.2 Conflit de processus Ollama

Un autre problème rencontré était le conflit entre plusieurs méthodes de lancement d'Ollama :

- service systemd ;
- service utilisateur ;
- lancement manuel avec `ollama serve`.

Le conflit provoquait l'erreur suivante :

```
bind: address already in use
```

La solution consiste à utiliser une seule méthode de lancement d'Ollama et à éviter de démarrer plusieurs processus simultanément.

9.3 Lenteur du modèle de génération

Le modèle `qwen3:8b` offre une meilleure qualité, mais peut être lent. Pour le développement quotidien, un modèle plus léger comme `qwen2.5:3b` peut être utilisé.

Pour la démonstration finale, `qwen3:8b` reste préférable si la machine peut le supporter.

9.4 Tokens JWT expirés

Un autre problème rencontré était l'expiration ou l'invalidation du token JWT. Dans ce cas, les endpoints protégés retournent :

Could not validate credentials

La solution est simplement de se reconnecter et de remplacer la variable `TOKEN`.

10 Prochaine étape : Phase 12

10.1 Objectif de la Phase 12

La prochaine phase consiste à ajouter une évaluation automatique des réponses générées.

L'objectif est de mesurer :

- la fidélité de la réponse au contexte ;
- la pertinence de la réponse ;
- le risque d'hallucination ;
- la qualité des chunks récupérés.

10.2 Pipeline prévu

```
Question utilisateur
|
v
Retrieval
|
v
Generation RAG
|
v
Evaluation automatique
|
v
Sauvegarde des scores
|
v
Affichage des metriques
```

10.3 Importance de cette phase

La Phase 12 est importante car elle permet de transformer le système en une plateforme plus fiable. Un système RAG professionnel ne doit pas seulement générer des réponses ; il doit aussi être capable d'évaluer leur qualité.

Conclusion

Les phases 9 à 11 représentent une étape majeure dans le développement de DevPilot AI. Le projet est passé d'un simple pipeline d'ingestion documentaire à une plateforme RAG fonctionnelle.

La Phase 9 a permis de générer des embeddings avec Ollama et de stocker les vecteurs dans Qdrant. La Phase 10 a ajouté la recherche sémantique avec filtrage par workspace. La Phase 11 a permis de construire un chat RAG capable de générer des réponses avec citations à partir des documents privés de l'utilisateur.

Le résultat obtenu montre que DevPilot AI possède maintenant les composants essentiels d'une plateforme moderne d'intelligence documentaire :

- stockage documentaire ;
- chunking ;
- embeddings ;
- vector search ;
- retrieval ;
- génération locale ;
- citations ;
- sécurité par workspace.

La prochaine étape consistera à ajouter l'évaluation automatique des réponses, puis un workflow agentique pour améliorer encore la qualité, la fiabilité et la contrôlabilité du système.

État actuel : Phase 11 validée – Chat RAG avec citations fonctionnel.

Prochaine étape : Phase 12 – Évaluation automatique des réponses.